

PROMPT MESTRE — ORGANIZAÇÃO UX/UI, SIDEBAR, DASHBOARDS E PERFIS DO DATAQUEST

CONTEXTO DO PROJETO

Você está trabalhando sobre o projeto existente **DataQuest — Sistema de Gestão de Pesquisas**.

O projeto já possui funcionalidades implementadas para gestão de pesquisas/questionários, coleta de respostas, pesquisadores de campo, metas, análise, importação, dimensionamento, colaboradores, perfis de acesso, auditoria, sincronização offline e demais recursos existentes no código.

O projeto utiliza atualmente uma arquitetura baseada em:

- React;
- TypeScript;
- Vite;
- Tailwind CSS;
- Supabase;
- Vercel;
- PWA/offline;
- Context API;
- componentes React existentes;
- serviços existentes;
- APIs existentes;
- banco Supabase existente;
- sistema de perfis e permissões existente.

OBJETIVO DESTA TAREFA

Quero fazer uma **reorganização completa da experiência visual e da navegação do DataQuest**, sem reconstruir o sistema.

O objetivo é tornar a aplicação:

- mais organizada;
- mais intuitiva;
- mais rápida de utilizar;
- mais profissional;
- mais clara;
- mais fácil de aprender;
- mais adequada para cada perfil;
- melhor em desktop;
- melhor em tablet;
- melhor em celular;
- com menos poluição visual;
- com menos cliques desnecessários;
- com informações apresentadas conforme o contexto do usuário.

REGRA ABSOLUTA

NÃO CRIE NOVAS FUNCIONALIDADES.

Esta tarefa é de:

UX + UI + organização + navegação + hierarquia + tradução + experiência por perfil.

Não é uma tarefa para ampliar o produto.

Não crie:

- novos módulos;
- novos processos;
- novas entidades;
- novas tabelas;
- novas APIs;
- novas integrações;
- novos fluxos de negócio;
- novos tipos de usuário;
- novos sistemas de autenticação;
- novos sistemas de permissões;
- novas funcionalidades de pesquisa;
- novas funcionalidades de coleta;
- novas funcionalidades de análise.

Se algo não existe atualmente, não implemente.

Se uma melhoria puder ser realizada apenas reorganizando algo que já existe, faça.

1. ANTES DE ALTERAR O CÓDIGO

Leia o projeto inteiro e compreenda sua arquitetura.

Analise especialmente:

- `src/App.tsx`
- `src/i18n.ts`
- `src/types.ts`
- `src/context/AppContext.tsx`
- `src/components/Sidebar.tsx`
- `src/components/Header.tsx`
- `src/components/HomeDashboard.tsx`
- `src/components/surveys/SurveyList.tsx`
- `src/components/wizard/SurveyWizard.tsx`
- `src/components/responses/ResponsesModule.tsx`

- `src/components/analytics/AnalyticsModule.tsx`
- `src/components/metastats/MetasModule.tsx`
- `src/components/metastats/MobileMetasDashboard.tsx`
- `src/components/metastats/ResearcherIndividualGoalsView.tsx`
- `src/components/team/TeamSizingModule.tsx`
- `src/components/import/ExternalImportModule.tsx`
- `src/components/registrations/CollaboratorForm.tsx`
- `src/components/registrations/AccessPolicies.tsx`
- `src/components/researcher/ResearcherEnvironment.tsx`
- `src/components/audit/ActionHistory.tsx`
- `src/components/simulator/CollectionSimulator.tsx`
- componentes de PWA;
- componentes de sincronização;
- serviços Supabase;
- serviços de sincronização;
- APIs;
- tipos de usuários;
- permissões;
- dados mock existentes;
- estrutura do banco;
- migrations;
- seed.

Também analise o `README.md`.

2. MAPEIE O SISTEMA EXISTENTE

Antes de implementar qualquer alteração, identifique:

Módulos existentes

- Dashboard/Home
- Pesquisas
- Criação/Edição de Pesquisa
- Respostas
- Análise
- Metas
- Dimensionamento
- Importação
- Colaboradores
- Políticas de Acesso
- Auditoria
- Simulador
- Ambiente do Pesquisador
- demais módulos efetivamente existentes.

Não invente módulos.

3. PERFIS

O sistema já possui perfis e políticas de acesso.

Não crie novos perfis.

Analise os perfis existentes no código e organize a interface conforme a função real de cada um.

A experiência deve ser diferente para cada perfil.

O princípio deve ser:

"O usuário não deve navegar pelo sistema inteiro; ele deve navegar pelo trabalho que precisa executar."

4. PRINCIPAIS PERFIS A CONSIDERAR

A estrutura atual possui, entre outros, perfis relacionados a:

- Administração;
- Coordenação/Gestão;
- Pesquisador de campo;
- Analista;
- demais perfis efetivamente definidos no projeto.

NÃO assuma que esses são os únicos perfis.

Leia `types.ts`, `AppContext.tsx`, dados iniciais e políticas de acesso para descobrir os perfis reais.

Preserve os IDs internos.

Você pode melhorar o nome apresentado na interface, mas não altere identificadores internos se isso puder quebrar o sistema.

5. CADA PERFIL DEVE TER UMA EXPERIÊNCIA PRÓPRIA

Este é um dos requisitos mais importantes.

Não quero simplesmente um Sidebar único com várias opções escondidas.

Quero que cada perfil tenha uma experiência contextual.

Por exemplo:

GESTÃO

Priorizar:

- visão geral;
- pesquisas;
- respostas;
- análise;
- metas;
- equipe;
- relatórios/recursos analíticos existentes;
- importação;
- colaboradores;
- políticas de acesso;
- auditoria.

PESQUISADOR DE CAMPO

Priorizar:

- minhas pesquisas;
- coleta;
- metas;
- progresso;
- histórico;
- sincronização;
- estado online/offline;
- recursos diretamente relacionados à coleta.

O pesquisador não precisa navegar por:

- administração;
- políticas;
- colaboradores;
- importação;
- análise administrativa;
- configurações que não sejam pertinentes.

ANALISTA

Priorizar:

- pesquisas autorizadas;
- respostas;
- análise;
- exportações existentes;
- informações necessárias para interpretação dos dados.

Não apresentar menus administrativos sem permissão.

ADMINISTRADOR

Priorizar:

- dashboard;
- pesquisas;
- equipe;
- colaboradores;
- políticas de acesso;
- auditoria;
- gestão;
- recursos administrativos existentes.

IMPORTANTE:

Esses exemplos são apenas diretrizes de UX.

Utilize os perfis e permissões que realmente existem no projeto.

6. SIDEBAR — NOVA ORGANIZAÇÃO

O Sidebar deve deixar de ser uma lista simples de módulos.

Organize por **blocos de tarefas**.

A organização deve ser contextual.

Uma possível estrutura para usuários administrativos:

VISÃO GERAL

- Início

PESQUISA

- Pesquisas
- Criar pesquisa, se permitido

COLETA

- Respostas
- Simulador, se permitido

ANÁLISE

- Análise de resultados
- Exportações existentes

METAS E PLANEJAMENTO

- Metas
- Dimensionamento

OPERAÇÃO

- Importação
- Equipe/Colaboradores

SEGURANÇA E CONTROLE

- Políticas de acesso
- Auditoria

Essa organização deve usar SOMENTE funcionalidades que já existem.

7. SIDEBAR DO PESQUISADOR

O pesquisador de campo deve ter um Sidebar completamente diferente.

Não mostrar o menu administrativo completo.

Priorizar algo como:

INÍCIO

- Meu painel

COLETA

- Minhas pesquisas
- Coleta

DESEMPENHO

- Minhas metas
- Meu histórico

SINCRONIZAÇÃO

- Sincronização
- Estado da conexão

CONFORMIDADE

- Trilha de conformidade, somente se permitido.

Use apenas funcionalidades existentes.

8. SIDEBAR DO ANALISTA

O analista deve ter uma interface orientada a dados.

Estrutura conceitual:

VISÃO GERAL

- Início

DADOS

- Pesquisas
- Respostas

ANÁLISE

- Análise de resultados

EXPORTAÇÃO

- recursos de exportação já existentes

Não invente novos recursos analíticos.

9. SIDEBAR DO ADMINISTRADOR

Organize:

VISÃO GERAL

- Início

PESQUISA

- Pesquisas

DADOS

- Respostas
- Análise

PLANEJAMENTO

- Metas
- Dimensionamento

EQUIPE

- Colaboradores

IMPORTAÇÃO

- Importação externa

SEGURANÇA

- Políticas de acesso
- Auditoria

Somente quando autorizado pelas permissões existentes.

10. REGRAS DO SIDEBAR

O Sidebar deve:

- agrupar por contexto;
 - mostrar títulos de seção;
 - manter hierarquia visual;
 - destacar a página atual;
 - permitir recolhimento quando fizer sentido;
 - funcionar bem no mobile;
 - respeitar permissões;
 - não mostrar opções sem acesso;
 - não duplicar funcionalidades;
 - evitar menus muito longos.
-

11. NÃO ALTERAR O SISTEMA DE PERMISSÕES

O projeto já possui uma estrutura de permissões granulares.

Preserve:

- `AccessPolicyPermissions`;
- `AccessProfile`;
- `hasPermission`;
- `profiles`;
- `updateProfile`;

- políticas existentes;
- regras existentes.

Não crie um RBAC paralelo.

Não substitua o atual por outro sistema.

12. PERMISSÃO DEVE CONTROLAR A NAVEGAÇÃO

Não basta esconder visualmente.

A navegação deve continuar protegida pelas permissões existentes.

Se o usuário não possuir:

`pesquisa_acesso`

não deve visualizar/acessar o módulo de pesquisa.

Se não possuir:

`analise_acesso`

não deve visualizar/acessar análise.

E assim por diante.

Preserve a lógica de segurança existente.

13. ÁREA DE CADASTRO E PERFIS

A área de colaboradores já existe.

Melhore sua organização visual.

Quero uma experiência mais clara para:

Lista de colaboradores

Mostrar de forma organizada:

- nome;
- login;
- perfil;
- status;
- pesquisas vinculadas;

- ações disponíveis.

Use os dados existentes.

14. CADASTRO DE COLABORADOR

O formulário existente possui diversos campos.

Não remova campos existentes.

Organize-os em grupos visuais:

IDENTIFICAÇÃO

- nome;
- CPF;
- RG;
- data de nascimento;
- sexo.

CONTATO

- e-mail;
- celular;
- telefone.

ACESSO

- login;
- senha;
- perfil de acesso;
- status.

PESQUISAS VINCULADAS

- pesquisas vinculadas já existentes.

Não crie novos campos.

15. POLÍTICAS DE ACESSO

A tela `AccessPolicies` já possui uma matriz de permissões bastante detalhada.

Não crie permissões novas.

Melhore a apresentação.

Organize visualmente por grupos:

Dashboard

Colaboradores

Pesquisas

Respostas

Análise

Metas

Importação

Políticas de acesso

Utilize exatamente as permissões existentes.

16. TELA DE PERFIL

Quando o administrador selecionar um perfil, a interface deve facilitar a compreensão.

Mostrar:

Nome do perfil

Descrição

Módulos acessíveis

Permissões disponíveis

Cada permissão deve apresentar:

- nome;
- descrição;
- controle visual;
- estado ativo/inativo.

O administrador deve compreender rapidamente:

"O que este perfil pode fazer?"

17. DASHBOARD

O dashboard atual possui muitos indicadores e informações.

Quero torná-lo mais limpo.

Não remover informações existentes sem necessidade.

Reorganizar.

Priorizar:

1 — Situação atual

O que está acontecendo agora.

2 — Pesquisas

Situação das pesquisas.

3 — Coletas

Quantidade de respostas/coletas.

4 — Metas

Progresso das metas existentes.

5 — Equipe

Informações existentes relacionadas à operação.

6 — Sincronização/Conexão

Somente quando pertinente.

18. NÃO TRANSFORMAR O DASHBOARD EM UM PAINEL DE TUDO

O dashboard deve responder:

"O que eu preciso saber agora?"

Não deve responder:

"Tudo que existe no sistema."

Reduza a competição visual.

Use:

- cards;
 - agrupamentos;
 - hierarquia;
 - informações resumidas;
 - ações rápidas existentes.
-

19. DASHBOARD POR PERFIL

O dashboard deve ser contextual.

Gestão

Mostrar principalmente:

- pesquisas;
- coletas;
- metas;
- operação;
- equipe;
- indicadores já existentes.

Pesquisador

O `ResearcherEnvironment` deve ser tratado como seu ambiente principal.

Priorizar:

- pesquisas atribuídas;
- coleta;
- progresso;
- metas;
- histórico;
- conexão;
- sincronização.

Analista

Priorizar:

- respostas;
- análises;
- indicadores;
- pesquisas disponíveis.

Não mostrar informações administrativas irrelevantes.

20. AMBIENTE DO PESQUISADOR

O componente `ResearcherEnvironment` já é especializado.

Não substitua sua funcionalidade.

Melhore sua hierarquia.

Organize as informações em uma ordem clara:

IDENTIDADE

Quem está conectado.

SITUAÇÃO DA CONEXÃO

- online;
- offline;
- sincronização.

Use os estados existentes.

MINHAS PESQUISAS

Mostrar apenas pesquisas às quais o pesquisador está vinculado, respeitando a lógica existente.

COLETA

Destacar a ação de coleta existente.

METAS

Mostrar progresso das metas existentes.

HISTÓRICO

Mostrar histórico existente.

SINCRONIZAÇÃO

Mostrar claramente o estado da fila e sincronização existente.

21. PESQUISAS

A tela de pesquisas deve facilitar a localização de uma pesquisa.

Organize:

- pesquisa;
- código;
- status;
- período;
- pesquisadores;
- ações disponíveis.

Utilize filtros existentes.

Não invente filtros novos se eles não existirem.

Pode melhorar visualmente os filtros existentes.

22. CRIAÇÃO/EDIÇÃO DE PESQUISA

O `SurveyWizard` já existe.

Não recrie o wizard.

Não altere sua lógica.

Melhore:

- hierarquia;
- títulos;
- etapas;
- espaçamento;
- agrupamento;
- indicação de progresso;
- ações de salvar/cancelar/avançar/voltar.

Não altere a lógica das perguntas.

Não altere as regras condicionais.

Não altere o modelo de dados.

23. RESPOSTAS

A tela de respostas deve ser orientada à consulta dos dados.

Melhorar:

- tabela;
- filtros existentes;
- identificação da pesquisa;
- status;
- informações principais;
- ações permitidas.

Não criar novos recursos de edição.

Preservar as permissões existentes.

24. ANÁLISE

O módulo de análise deve ter uma interface mais limpa.

Priorizar:

- pesquisa selecionada;
- indicadores;
- gráficos existentes;
- distribuição;
- NPS;
- demais análises que já existem.

Não adicionar novos indicadores.

Não inventar novos gráficos.

Apenas reorganizar os existentes.

25. METAS

Organizar a visualização das metas existentes.

Separar visualmente:

- meta;
- alvo;
- atingido;
- percentual;

- status.

Para o pesquisador:

mostrar somente suas metas e informações que ele já possui acesso.

Não criar novo sistema de metas.

26. DIMENSIONAMENTO

O módulo de dimensionamento já existe.

Preservar seus cálculos.

Melhorar somente:

- organização;
- legibilidade;
- apresentação dos parâmetros;
- resultados;
- ações.

Não alterar fórmulas.

Não alterar regras estatísticas.

Não alterar níveis de confiança existentes.

27. IMPORTAÇÃO

A área de importação externa deve continuar funcionando exatamente como está.

Organizar visualmente:

Arquivo

Mapeamento

Processamento

Resultado

Somente se essas etapas já estiverem representadas pelo fluxo existente.

Não criar novo fluxo de importação.

28. AUDITORIA

A tela de auditoria deve transmitir segurança e rastreabilidade.

Organizar:

- data;
- usuário;
- ação;
- registro;
- informações existentes.

Não alterar o mecanismo de auditoria.

Não remover informações.

Não inventar novos eventos.

29. SIMULADOR

O simulador existente deve continuar sendo tratado como uma ferramenta já existente.

Não transformá-lo em uma nova funcionalidade.

Melhorar apenas a navegação e apresentação.

30. IDIOMA — PORTUGUÊS DO BRASIL

Faça uma auditoria completa da interface.

Tudo que aparecer para o usuário deve estar em:

Português do Brasil.

Verifique:

- Sidebar;
- Header;
- Dashboard;
- botões;
- tooltips;
- mensagens;
- alertas;
- modais;
- formulários;

- placeholders;
 - tabelas;
 - status;
 - permissões;
 - textos de erro;
 - textos de sucesso;
 - mensagens offline;
 - sincronização;
 - PWA.
-

31. INTERNACIONALIZAÇÃO EXISTENTE

O projeto já possui `src/i18n.ts`.

Não substitua o sistema de tradução sem necessidade.

Analise como ele funciona.

Utilize o mecanismo existente.

Se houver chaves já existentes, aproveite-as.

Se houver textos hardcoded em inglês que precisam ser exibidos em português, faça a tradução de forma compatível com a arquitetura existente.

Não altere nomes internos de variáveis apenas para traduzir a interface.

32. PADRÃO BRASILEIRO

Use:

Data

`dd/mm/aaaa`

Hora

`HH:mm`

Números

formatação brasileira.

Percentuais

formatação brasileira.

Textos

Português brasileiro natural.

Evite traduções literais estranhas.

33. RESPONSIVIDADE

A aplicação precisa funcionar bem em:

- desktop;
- notebook;
- tablet;
- celular.

Não faça apenas uma redução da versão desktop.

No mobile:

- simplifique;
 - reorganize;
 - empilhe;
 - priorize ações;
 - aumente áreas de toque;
 - reduza informação secundária.
-

34. MOBILE DO PESQUISADOR

O pesquisador de campo é um dos usuários mais importantes no mobile.

O ambiente de coleta deve ser otimizado para:

- uso com uma mão;
- toque;
- leitura rápida;
- ambiente externo;
- conexão instável;
- modo offline existente;
- GPS existente;
- áudio existente.

Não alterar essas funcionalidades.

Apenas melhorar a interface.

35. NAVEGAÇÃO MOBILE

Quando o espaço for limitado:

- priorize as funções principais;
- use navegação inferior ou menu móvel existente, se já houver;
- mantenha a ação principal facilmente acessível;
- evite menus enormes.

Não criar uma arquitetura mobile completamente diferente sem necessidade.

36. CONSISTÊNCIA VISUAL

Faça o sistema parecer um único produto.

Padronize:

- cards;
- botões;
- inputs;
- selects;
- tabelas;
- badges;
- status;
- modais;
- espaçamento;
- títulos;
- subtítulos;
- ícones.

Preserve a identidade visual existente.

Não substitua a paleta inteira sem necessidade.

37. DARK MODE

Se o sistema já suporta modo escuro, preserve.

Garanta que as novas organizações visuais funcionem bem em:

- dark mode;
- light mode.

Não criar um terceiro tema.

38. ACESSIBILIDADE

Melhore quando possível:

- contraste;
- foco;
- navegação por teclado;
- tamanho das áreas clicáveis;
- labels;
- mensagens;
- hierarquia semântica;
- feedback visual.

Não altere a funcionalidade.

39. PERFORMANCE

Não introduza problemas de performance.

Evite:

- renders desnecessários;
- consultas duplicadas;
- chamadas duplicadas;
- carregamentos redundantes;
- cálculos repetidos.

Não faça uma grande refatoração arquitetural.

40. SUPABASE

Preserve integralmente:

- Supabase;
- tabelas;
- migrations;
- RLS;
- Storage;
- serviços;
- sincronização.

NÃO criar banco paralelo.

NÃO alterar schema apenas para realizar redesign visual.

41. VERCCEL

Preserve:

- APIs;
- serverless functions;
- configuração;
- build;
- deploy.

Não alterar infraestrutura.

42. PWA / OFFLINE

O sistema já possui recursos de PWA e sincronização offline.

Não remover.

Não quebrar.

Não substituir.

Não alterar o comportamento de sincronização apenas para modificar a interface.

Apenas melhorar a apresentação dos estados:

- online;
- offline;
- pendente;
- sincronizando;
- sincronizado;
- erro.

Use os estados que já existem.

43. ÁUDIO E GEOLOCALIZAÇÃO

O sistema já trabalha com:

- gravação/áudio;
- reprodução;
- georreferenciamento;
- mapas;
- localização de coleta.

Não modificar a lógica dessas funcionalidades.

Melhorar somente a forma como elas são apresentadas e acessadas.

44. SEGURANÇA

Não exponha informações para perfis que não possuem permissão.

A reorganização visual não pode criar brechas.

Não confiar apenas na interface para segurança.

Preservar:

- permissões;
 - validações;
 - RLS;
 - regras existentes.
-

45. NÃO CONFUNDIR "OCULTAR" COM "PROTEGER"

Se um menu desaparecer para determinado perfil, o módulo ainda deve continuar protegido pelo sistema existente.

Não alterar o modelo de segurança.

46. MICROCOPY

Melhore textos quando estiverem:

- técnicos demais;
- confusos;
- em inglês;
- inconsistentes.

Use português natural.

Exemplo:

Em vez de:

"Access Policies"

usar:

"Políticas de acesso"

Em vez de:

"Responses"

usar:

"Respostas"

Em vez de:

"Researcher Environment"

usar:

"Ambiente do Pesquisador"

Mas preserve nomes técnicos internos.

47. ESTADOS VAZIOS

Revise telas existentes para melhorar estados vazios.

Exemplos:

- nenhuma pesquisa;
- nenhuma resposta;
- nenhuma meta;
- nenhuma pesquisa vinculada;
- nenhuma sincronização pendente.

Mensagens devem ser claras.

Não inventar novas ações.

48. AÇÕES RÁPIDAS

As ações mais utilizadas devem ficar próximas das informações relacionadas.

Por exemplo:

Na tela de pesquisa:

- abrir;
- editar, quando permitido;

- iniciar ações existentes.

No ambiente do pesquisador:

- iniciar coleta;
- consultar metas;
- consultar histórico;
- sincronizar.

Não criar ações que não existam.

49. REGRA DOS POUCOS CLIQUES

Sempre que reorganizar uma tela, pergunte:

"Quantos cliques o usuário precisa para executar a tarefa?"

Tente reduzir etapas sem alterar o fluxo de negócio.

Não esconda funcionalidades atrás de vários menus.

Não aumente a profundidade de navegação.

50. HIERARQUIA DE INFORMAÇÃO

Em cada tela, estabeleça:

PRINCIPAL

O que o usuário veio fazer.

SECUNDÁRIO

Informações que ajudam na tarefa.

CONTEXTO

Informações úteis, mas não prioritárias.

ADMINISTRATIVO

Opções menos frequentes.

Evite que tudo tenha o mesmo peso visual.

51. EVITAR EXCESSO DE CARDS

Não transforme toda informação em card.

Use cards apenas quando ajudarem na leitura.

Evite:

- card dentro de card;
 - excesso de bordas;
 - excesso de sombras;
 - excesso de cores;
 - excesso de badges.
-

52. EVITAR DASHBOARD "SHOWCASE"

O dashboard não deve parecer uma tela criada para mostrar tudo que o sistema consegue fazer.

Ele deve ser operacional.

O usuário precisa olhar e entender rapidamente:

- o que aconteceu;
 - o que está acontecendo;
 - o que precisa de atenção;
 - onde clicar.
-

53. PERFIL COMO CONTEXTO

O perfil do usuário deve influenciar:

- Sidebar;
- dashboard;
- ações rápidas;
- conteúdo prioritário;
- informações visíveis;
- navegação.

Não criar páginas duplicadas desnecessariamente.

Reutilize componentes quando possível, adaptando a apresentação.

54. NÃO DUPLICAR CÓDIGO SEM NECESSIDADE

Se duas experiências utilizarem o mesmo componente funcional, mantenha o componente reutilizável.

Pode criar componentes de apresentação específicos quando necessário.

Mas não replique lógica de negócio.

55. ARQUITETURA

Não introduza:

- novo framework;
- novo sistema de estado;
- novo sistema de rotas;
- nova biblioteca UI;
- novo sistema de autenticação;
- novo sistema RBAC;
- nova biblioteca de internacionalização sem necessidade.

Primeiro use o que já existe.

56. TYPESCRIPT

Preserve a tipagem.

Não use:

`any`

como solução fácil para erros introduzidos pela reorganização.

Não altere interfaces existentes sem necessidade.

57. TESTES E BUILD

Após as alterações:

Execute:

- TypeScript;
- lint;
- build.

Corrija erros introduzidos.

Não considere a tarefa concluída se o build quebrar.

58. TESTE DE PERFIS

Faça uma revisão simulando cada perfil existente.

Para cada perfil, verifique:

O que aparece no Sidebar?

O que aparece no Dashboard?

Quais módulos podem ser acessados?

Quais ações aparecem?

Existem opções que não deveriam aparecer?

Existe algum acesso que desapareceu indevidamente?

59. TESTE DO PESQUISADOR

Simule especialmente o pesquisador de campo.

Verifique:

- login;
- ambiente do pesquisador;
- pesquisas vinculadas;
- coleta;
- metas;
- histórico;
- offline;
- sincronização;
- GPS;
- áudio.

Não quebrar nenhum desses fluxos.

60. TESTE MOBILE

Faça uma revisão em viewport pequeno.

Verifique:

- Sidebar;
- menu mobile;
- dashboard;
- tabelas;
- modais;
- formulários;
- coleta;
- metas;
- sincronização.

Nada deve gerar overflow horizontal desnecessário.

61. O QUE NÃO DEVE SER ALTERADO

Não alterar sem necessidade:

- banco;
 - migrations;
 - RLS;
 - APIs;
 - autenticação;
 - serviços;
 - modelos de dados;
 - regras de negócio;
 - fórmulas estatísticas;
 - regras condicionais;
 - lógica de coleta;
 - sincronização;
 - armazenamento;
 - áudio;
 - geolocalização.
-

62. OBSERVAÇÃO IMPORTANTE SOBRE AUTENTICAÇÃO

O próprio projeto documenta que a autenticação atual possui uma implementação que ainda não utiliza o Supabase como fonte definitiva para validação da senha.

Isso é uma questão de arquitetura/segurança.

NÃO CORRIJA ISSO NESTA TAREFA.

Não transformar este trabalho em uma migração de autenticação.

Não criar `api/auth/login.ts`.

Não alterar a autenticação.

O escopo aqui é UX/UI e organização.

63. DADOS MOCK

O projeto possui `mockData.ts`.

Não substitua dados reais por mock.

Não crie dados fictícios para "preencher" a interface.

Se o projeto utilizar dados demonstrativos atualmente, preserve o comportamento existente.

64. PRINCÍPIO DE NÃO REGRESSÃO

Uma melhoria visual não pode destruir uma funcionalidade.

Depois da reorganização:

- toda pesquisa continua abrindo;
 - wizard continua funcionando;
 - respostas continuam acessíveis;
 - análise continua funcionando;
 - metas continuam funcionando;
 - dimensionamento continua funcionando;
 - importação continua funcionando;
 - colaboradores continuam funcionando;
 - políticas continuam funcionando;
 - auditoria continua funcionando;
 - simulador continua funcionando;
 - coleta continua funcionando;
 - sincronização continua funcionando.
-

65. PROCESSO DE EXECUÇÃO

Siga esta ordem.

FASE 1 — AUDITORIA

Mapeie:

- componentes;
- módulos;
- perfis;
- permissões;
- navegação;
- serviços;
- dados.

FASE 2 — MAPA DE NAVEGAÇÃO

Crie mentalmente uma matriz:

Perfil	Dashboard	Pesquisas	Respostas	Análise	Metas	Equipe	Importação	Auditoria
--------	-----------	-----------	-----------	---------	-------	--------	------------	-----------

Preencha com base nas permissões reais.

FASE 3 — SIDEBAR

Reorganize por tarefas.

FASE 4 — DASHBOARD

Simplifique e contextualize.

FASE 5 — PERFIS

Ajuste a experiência de cada perfil.

FASE 6 — CADASTRO/PERMISSÕES

Melhore a organização.

FASE 7 — IDIOMA

Português brasileiro completo.

FASE 8 — RESPONSIVIDADE

Desktop + tablet + mobile.

FASE 9 — VALIDAÇÃO

Build + TypeScript + revisão funcional.

66. CRITÉRIO PARA DECIDIR SE UMA ALTERAÇÃO É PERMITIDA

Antes de cada alteração, pergunte:

1. A funcionalidade já existe?
2. Estou somente reorganizando?
3. Estou somente melhorando UX/UI?
4. Estou preservando os dados?
5. Estou preservando a API?
6. Estou preservando as permissões?
7. Estou preservando o fluxo?
8. Estou preservando o banco?
9. Estou preservando a autenticação?
10. Estou preservando o comportamento offline?

Se alguma resposta for NÃO:

não implemente automaticamente.

67. RESULTADO FINAL ESPERADO

Quero que o DataQuest passe a transmitir a seguinte sensação:

ADMINISTRADOR

"Consigo controlar o sistema rapidamente."

GESTOR

"Consigo entender o andamento das pesquisas rapidamente."

PESQUISADOR

"Consigo iniciar minha coleta e acompanhar minhas metas sem procurar as funções."

ANALISTA

"Consigo chegar aos dados e análises rapidamente."

USUÁRIO NOVO

"Entendi onde estão as coisas sem precisar de treinamento."

68. PRINCÍPIO CENTRAL DO DESIGN

A interface deve seguir:

menos navegação → mais contexto → menos informação irrelevante → ações mais rápidas.

Não queremos necessariamente menos funcionalidades.

Queremos:

menos complexidade para chegar às funcionalidades existentes.

69. ENTREGA

Ao terminar:

1. atualize o código;
 2. não remova funcionalidades;
 3. não crie funcionalidades novas;
 4. mantenha a arquitetura;
 5. mantenha as integrações;
 6. mantenha o banco;
 7. mantenha as permissões;
 8. mantenha a coleta;
 9. mantenha offline/PWA;
 10. mantenha áudio;
 11. mantenha geolocalização;
 12. mantenha sincronização;
 13. traduza a interface para português;
 14. melhore desktop;
 15. melhore mobile;
 16. faça o Sidebar contextual;
 17. faça dashboards contextuais;
 18. organize cadastro de usuários;
 19. organize políticas de acesso;
 20. valide o build.
-

70. RELATÓRIO FINAL

Ao finalizar, apresente um resumo contendo:

Navegação

Quais grupos foram criados no Sidebar.

Perfis

Como cada perfil passou a enxergar o sistema.

Dashboard

Quais informações foram priorizadas.

Mobile

Quais melhorias foram feitas.

Idioma

Quais áreas foram traduzidas.

Permissões

Como o RBAC existente foi preservado.

Segurança

Confirme que não houve alteração indevida.

Funcionalidades

Confirme que nenhuma funcionalidade existente foi removida.

Infraestrutura

Confirme que:

- Supabase;
- Vercel;
- APIs;
- PWA;
- sincronização;

foram preservados.

INSTRUÇÃO FINAL

Leia o ZIP inteiro antes de começar.

Não responda com uma proposta genérica.

Analise o código real.

Identifique os perfis reais.

Identifique as permissões reais.

Identifique os módulos reais.

Identifique o fluxo real de navegação.

Depois faça a reorganização.

NÃO RECONSTRUA O DATAQUEST.

NÃO CRIE UM NOVO SISTEMA.

NÃO MUDE O BANCO.

NÃO MUDE O SUPABASE.

NÃO MUDE A AUTENTICAÇÃO.

NÃO MUDE A API.

NÃO MUDE AS REGRAS DE NEGÓCIO.

NÃO CRIE FUNCIONALIDADES NOVAS.

NÃO REMOVA FUNCIONALIDADES EXISTENTES.

Faça uma **refatoração de experiência de usuário e organização visual sobre o sistema existente.**

O resultado deve ser um DataQuest mais limpo, profissional, intuitivo, responsivo e contextual.

Cada perfil deve sentir que está utilizando uma aplicação feita especificamente para seu trabalho, mesmo que por baixo dos panos continuemos utilizando os mesmos módulos, serviços, dados e regras existentes.

Comece pela auditoria. Depois apresente mentalmente o mapa de navegação por perfil. Em seguida implemente as mudanças de forma incremental, segura e sem regressão.